

Lernzettel - Ausgaben-Tracker

Cedric Conrads - JavaScript-Grundlagen anhand deines Projekts. Nutze diesen Zettel zum Nachschlagen, Wiederholen und Erklären in eigenen Worten.

Grundidee der App

Begriff / Code	Bedeutung
HTML	stellt Eingabefelder, Button, Summe und Liste bereit.
CSS	macht das Design: Karten, Farben, Abstände, Buttons.
JavaScript	holt Eingaben, prüft sie, rechnet, erstellt Listeneinträge und speichert Daten.

Wichtige HTML-IDs

Begriff / Code	Bedeutung
expense-name	Eingabefeld für den Namen der Ausgabe, z. B. Einkaufen.
expense-amount	Eingabefeld für den Betrag, z. B. 25.
add-btn	Button, den JavaScript findet und auf Klick überwacht.
total	Bereich, in dem die Gesamtsumme angezeigt wird.
expense-list	Bereich, in dem alle Ausgaben als Liste angezeigt werden.

Elemente suchen und speichern

Begriff / Code	Bedeutung
getElementById	sucht ein HTML-Element über seine ID.
<code>const expenseNameInput = document.getElementById("expense-name");</code>	JavaScript sucht das Element mit der ID expense-name und speichert es in der Variable expenseNameInput.
ID vs Variable	ID = Name im HTML. Variable = Name in JavaScript.

Variablen

Begriff / Code	Bedeutung
const	erstellt eine Variable, die nicht neu zugewiesen werden soll.
let	erstellt eine Variable, die später verändert werden darf.
<code>let total = 0;</code>	Die Gesamtsumme startet bei 0 und darf sich später ändern.

Eingaben holen

Begriff / Code	Bedeutung
.value	holt den Inhalt aus einem Eingabefeld.
.trim()	entfernt Leerzeichen vorne und hinten.
Number(...)	wandelt einen Text in eine Zahl um, damit JavaScript rechnen kann.

Begriff / Code	Bedeutung
<code>const expenseName = expenseNameInput.value.trim();</code>	Hole den Namen aus dem Eingabefeld, entferne Leerzeichen und speichere ihn.
<code>const expenseAmount = Number(expenseAmountInput.value);</code>	Hole den Betrag aus dem Eingabefeld und mache daraus eine Zahl.

Pruefung mit if

Begriff / Code	Bedeutung
<code>if</code>	wenn eine Bedingung stimmt.
<code>===</code>	ist genau gleich.
<code>""</code>	leerer Text.
<code> </code>	oder.
<code><=</code>	kleiner oder gleich.
<code>return</code>	stoppt die Funktion an dieser Stelle.
<code>if (expenseName === "" expenseAmount <= 0)</code>	Wenn der Name leer ist ODER der Betrag 0 oder kleiner ist.

Rechnen und Anzeigen

Begriff / Code	Bedeutung
<code>total = total + expenseAmount;</code>	Nimm die aktuelle Summe und addiere den neuen Betrag dazu.
<code>totalText.textContent = total;</code>	Zeige die neue Summe auf der Webseite an.
<code>textContent</code>	aendert den Textinhalt eines Elements.

Neue Elemente erstellen

Begriff / Code	Bedeutung
<code>document.createElement("li")</code>	erstellt einen neuen Listenpunkt.
<code>const listItem = document.createElement("li");</code>	Erstelle einen neuen Listenpunkt und speichere ihn in listItem.
<code>className</code>	setzt eine CSS-Klasse per JavaScript.
<code>listItem.className = "expense-item";</code>	Der neue Listenpunkt bekommt die CSS-Klasse expense-item.
<code>innerHTML</code>	schreibt HTML-Inhalt in ein Element hinein.
<code>appendChild</code>	haengt ein Element in ein anderes Element ein.
<code>expenseList.appendChild(listItem);</code>	Haenge den neuen Listenpunkt in die Ausgabenliste ein.

Loeschen-Button

Begriff / Code	Bedeutung
<code>querySelector</code>	sucht ein Element mit CSS-Schreibweise.
<code>querySelector(".delete-btn")</code>	sucht ein Element mit der Klasse delete-btn.

Begriff / Code	Bedeutung
<code>const deleteBtn = listItem.querySelector(".delete-btn");</code>	Suche innerhalb dieses Listeneintrags den Loeschen-Button und speichere ihn.
<code>remove()</code>	entfernt ein Element von der Webseite.
<code>listItem.remove();</code>	entfernt diesen Listeneintrag von der Webseite.

Arrays und Objekte

Begriff / Code	Bedeutung
Array	eine Liste mit mehreren Werten.
<code>[]</code>	leere Liste.
<code>let expenses = [];</code>	Erstelle eine leere Liste fuer alle Ausgaben.
<code>push</code>	fuegt etwas zu einer Liste hinzu.
<code>expenses.push(...)</code>	fuegt eine neue Ausgabe zur expenses-Liste hinzu.
<code>{}</code>	Objekt: zusammengehoerige Daten, z. B. Name und Betrag.
<code>{ name: expenseName, amount: expenseAmount }</code>	Eine Ausgabe mit Name und Betrag.

localStorage

Begriff / Code	Bedeutung
<code>localStorage</code>	lokaler Speicher im Browser. Bleibt nach Neuladen erhalten, bis man ihn loescht.
<code>setItem</code>	speichern.
<code>getItem</code>	holen/laden.
<code>JSON.stringify</code>	Daten einpacken, damit localStorage sie speichern kann.
<code>JSON.parse</code>	gespeicherte Daten wieder auspacken und fuer JavaScript nutzbar machen.
<code>localStorage.setItem("expenses", JSON.stringify(expenses));</code>	Speichere die Ausgabenliste im Browser unter dem Namen expenses.
<code>JSON.parse(localStorage.getItem("expenses")) []</code>	Lade die gespeicherte Liste. Wenn nichts da ist, nimm eine leere Liste.

renderExpenses und forEach

Begriff / Code	Bedeutung
<code>renderExpenses()</code>	Funktion, die alle Ausgaben aus der Liste auf der Webseite anzeigt.
<code>forEach</code>	geht jeden Eintrag einer Liste einzeln durch.
<code>expenses.forEach(function (expense) { ... })</code>	Gehe jede Ausgabe in der expenses-Liste einzeln durch.
<code>expense</code>	eine einzelne Ausgabe aus der Liste.
<code>expenseList.innerHTML = "";</code>	Leere die sichtbare Liste zuerst, damit sie sauber neu aufgebaut werden kann.

Funktionen und Events

Begriff / Code	Bedeutung
function	gespeicherter Codeblock mit Namen.
saveExpenses()	Funktion jetzt ausführen.
addEventListener	wartet auf eine Aktion, z. B. Klick.
"click"	das Ereignis ist ein Klick.
deleteBtn.addEventListener("click", function () { ... })	Wenn der Loeschen-Button geklickt wird, fuehre diesen Code aus.

Mini-Ablauf: Ausgabe hinzufuegen

1. User schreibt Name und Betrag in die Eingabefelder.
2. JavaScript holt die Werte mit `.value`.
3. `trim()` entfernt Leerzeichen beim Namen.
4. `Number()` macht aus dem Betrag eine Zahl.
5. `if` prueft: Name leer oder Betrag ungueltig?
6. Wenn ungueltig: Fehlermeldung anzeigen und `return` stoppt die Funktion.
7. Wenn gueltig: Betrag zur Gesamtsumme addieren.
8. Neuen Listenpunkt mit `createElement` erstellen.
9. CSS-Klasse mit `className` setzen.
10. Inhalt mit `innerHTML` reinpacken.
11. Eintrag mit `appendChild` sichtbar in die Liste setzen.
12. Eingabefelder wieder leeren.
13. Ausgabenliste mit `localStorage` speichern.

Sehr kurze Merksaetze

- `id` = Name im HTML
- `class` = Name fuer CSS
- `const` = bleibt gleich
- `let` = darf sich aendern
- `value` = Inhalt aus Eingabefeld
- `textContent` = Text aendern
- `createElement` = neues Element erstellen
- `appendChild` = Element einhaengen
- `remove` = Element entfernen
- `push` = zur Liste hinzufuegen
- `forEach` = jeden Eintrag durchgehen
- `localStorage` = Browser-Speicher
- `setItem` = speichern
- `getItem` = holen
- `stringify` = einpacken
- `parse` = auspacken